

Fixing Donkey Kong Ladder Bug

<https://gemini.google.com/app/96ff05e9cee830e3>

User prompt: # Donkey Kong Repair Lab This project is a single-file Donkey Kong-lite clone using **Pygame**. It introduces students to sloped-platform physics, ladder climbing, and gravity using a small, readable object-oriented codebase. --- ## What's Provided A working Donkey Kong-lite game with: - A player-controlled climber with jumping and gravity that follows sloped platforms - Ladders connecting each platform level that the player can climb - Barrels that roll down sloped platforms and occasionally descend a ladder to the level below - Lives, scoring, and a win condition for reaching the princess It has **one deliberate bug** and **three optional features** left as empty functions. You are expected to **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional and more interesting. ### Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab. --- ## Getting Started ### Setup 1. Make sure you have Python 3.10+ installed. 2. Install dependencies: `bash pip install pygame` 3. Run the game: `bash python game.py` --- ## Controls: Left/Right to move, Space to jump, Up/Down to grab a ladder while standing near it, 'R' to reset. --- ## Tasks to Complete Each task must be completed using an iterative process involving LLM suggestions and your critical code review. ### Task 1: Fix the ladder-descent bug > Barrels are supposed to take a ladder down to the next platform only occasionally, so a few keep rolling across the top platform while others trickle down toward the player. In the current build, almost every barrel immediately takes the first ladder it reaches, so the lower platforms flood with barrels and the top platform empties out fast. Find the probability check that controls this in the `Barrel` class and correct it so barrels descend roughly 3 times out of 10, not roughly 7 times out of 10. ### Task 2: Implement `theme_color(score)` > Called once per frame in `draw_scene` as `screen.fill(theme_color(score) or BG)`, where `BG` is the default navy background `(15, 15, 25)`. It receives the player's current integer score and should return an `(r, g, b)` tuple to use as the background color, or `None` to keep the default. Idea: shift toward a warmer color as the score climbs, or briefly flash a color right after a big score gain. ### Task 3: Implement `on_barrel_jumped(player, barrel)` > Called from the main loop exactly once, on the first frame the player is airborne directly above a barrel — right after the 100-point jump bonus is awarded for that barrel. It receives the `Player` and the `Barrel` that was jumped. Its return value is ignored; it exists purely for side effects. Idea: spawn a short-lived floating "+100" label at the barrel's position, or play a sound. ### Task 4: Implement `score_multiplier(score)` > Called right before the barrel-jump bonus is added, as `score += int(100 * (score_multiplier(score) or 1))`. It receives the score *before* the bonus is added and should return a numeric multiplier, or `None` for the default 1x. Idea: return `2` once the score passes some threshold, to reward late-game play. --- ## Expected Behavior - The player follows the slope of each platform instead of floating above it or sinking into it - The player can only climb a ladder while standing near that ladder's x-position, on the platform range it connects - Barrels roll down slopes and occasionally (about 30% of the time) descend a ladder instead of continuing to roll along the platform - Barrels never fall through platforms - Reaching the princess ends the game with a win message; running out of lives (from barrel collisions) ends it with a game-over message --- ## Folder Structure `donkey_kong/ | game.py | README.md` --- ## Submission Checklist Submission is only the following three things: - [] A 10-second video of gameplay **before** your changes, showing the bug/broken behavior - [] A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working - [] The Chat/LLM used page link, with the complete chat history fix the ladder bug

Response: To fix the ladder-descent bug (Task 1), locate the probability check inside the `Barrel` class in `game.py` where it checks `random.random() < 0.7` and change the threshold from `0.7` (70%) to `0.3` (30%).

Here is the corrected portion of the `Barrel` class update method:

```
for index, (lx, _, upper) in enumerate(LADDERS):
    if upper == self.plat and abs(self.pos.x - lx) < 3 and index not in self.skip:
        self.skip.add(index)
        if random.random() < 0.3: # Changed from 0.7 to 0.3 (30% chance)
            self.ladder = index
            self.pos.x = lx
```

What this changes:

- **Before:** `0.7` meant barrels had a 70% chance of taking the first ladder they encountered, causing them to cascade downward immediately and flooding the bottom platforms.
- **After:** `0.3` sets the chance to 30%, allowing most barrels to continue rolling across the platforms while a fair portion trickle down as intended.

User prompt: implement task 2

Response: To implement **Task 2**, we need to fill out the `theme_color(score)` function in `game.py`.

As the score climbs, this implementation dynamically shifts the background color away from the default navy `(15, 15, 25)` toward a warmer, dusk-like crimson/purple gradient as you progress through the game.

Code Implementation

Replace the empty `theme_color(score)` function in `game.py` with the following:

```
def theme_color(score):
    """Return an (r, g, b) background colour for the current score, or None for the default."""
    if score <= 0:
        return None
    # Gradually shift toward a warmer, atmospheric tone as the score climbs
    progress = min(score / 1500.0, 1.0)
    r = int(15 + progress * 65) # Increases red tone
    g = int(15 + progress * 5)  # Keeps green low
    b = int(25 + progress * 15) # Slight shift in blue/purple
    return (r, g, b)
```

How it works:

- **At 0 Score:** Returns `None`, defaulting to the standard navy background `(15, 15, 25)`.
- **As Score Increases:** The background smoothly warms up toward a deep dusk/crimson hue, capping out at a maximum shift once the score reaches 1,500 points.

User prompt: implement task 3

Response: To implement **Task 3**, we can define a floating text effect system that spawns a temporary "+100" label right above the barrel whenever the player successfully jumps over it.

Code Implementation

To support floating text effects across frames, we need a small global list to track active labels, update them as they drift upward, and render them in `draw_scene`.

Here are the updates to make in your `game.py` file:

1. Add the Global List and Update `on_barrel_jumped`

Place `FLOATING_TEXTS` near the top configuration constants and implement the function:

```
FLOATING_TEXTS = []
```

```
def on_barrel_jumped(player, barrel):
    """Called when the player clears a barrel; add a bonus effect here."""
    FLOATING_TEXTS.append({
        'text': "+100",
        'pos': pygame.Vector2(barrel.pos.x, barrel.pos.y - 15),
        'timer': 0.8 # Lasts for 0.8 seconds
    })
```

2. Update `draw_scene` to Render Floating Labels

Add the rendering loop inside `draw_scene`:

```
def draw_scene(screen, font, player, barrels, score, lives, state):
    screen.fill(theme_color(score) or BG)
    # ... (existing drawing code for platforms, ladders, kong, princess, barrels, player, hud) ...

    # Render floating text effects (+100 jump bonuses)
    for ft in FLOATING_TEXTS:
        label = font.render(ft['text'], True, (255, 255, 0))
        screen.blit(label, label.get_rect(center=ft['pos']))

    # ... (rest of draw_scene for win/game over states)
```

3. Update the `main()` Game Loop

Inside the `main()` function, make sure to update active floating texts, make them drift upward, and clear them on reset:

- In the reset handler (`pygame.K_r`):

```
elif event.type == pygame.KEYDOWN and event.key == pygame.K_r:
    player.reset()
    barrels.clear()
    FLOATING_TEXTS.clear()
    score, lives, state = 0, 3, "play"
```

- Inside the `state == "play"` loop update block:

```
# Update floating text timers and positions
for ft in FLOATING_TEXTS[:]:
    ft['timer'] -= dt
    ft['pos'].y -= 30 * dt # Drift upward
    if ft['timer'] <= 0:
        FLOATING_TEXTS.remove(ft)
```

User prompt: the last snippet is right below the `if` statement?

Response: Yes, exactly! That snippet goes inside the `if state == "play":` block in your `main()` loop, right alongside where the barrels and player are updated.

Here is how the complete `if state == "play":` block in `main()` should look with the floating text updates included:

```
if state == "play":
    player.update(dt, pygame.key.get_pressed())
    spawn_timer -= dt
    if spawn_timer <= 0:
        barrels.append(Barrel())
        spawn_timer = random.uniform(1.8, 3.2)

    for barrel in barrels:
```

```

        barrel.update(dt)
        hit_range = BARREL_R + PLAYER_W / 2
        if player.center().distance_squared_to(barrel.pos) < hit_range ** 2:
            lives -= 1
            player.reset()
            barrels.clear()
            FLOATING_TEXTS.clear() # Clear texts on death/reset too
            state = "play" if lives > 0 else "lose"
            break
        above = 0 < barrel.pos.y - player.pos.y + BARREL_R < 40
        if not player.on_ground and above and abs(barrel.pos.x - player.pos.x) < 12 and not barrel.scored:
            barrel.scored = True
            score += int(100 * (score_multiplier(score) or 1))
            on_barrel_jumped(player, barrel)

    barrels[:] = [b for b in barrels if b.pos.y < HEIGHT + 30]

    # --- UPDATE FLOATING TEXTS HERE ---
    for ft in FLOATING_TEXTS[:]:
        ft['timer'] -= dt
        ft['pos'].y -= 30 * dt # Drift upward
        if ft['timer'] <= 0:
            FLOATING_TEXTS.remove(ft)
    # -----

    if player.center().distance_to(pygame.Vector2(PRINCESS_POS)) < 24:
        score += 1000
        state = "win"

```

User prompt: implement task 4

Response: To implement **Task 4**, we can fill out the `score_multiplier(score)` function in `game.py`. This function will dynamically reward late-game play by increasing the score multiplier for jumping over barrels as your score climbs.

Code Implementation

Replace the empty `score_multiplier(score)` function in your code with the following:

```

def score_multiplier(score):
    """Return a multiplier applied to points earned from clearing a barrel, or None for the default 1x."""
    if score >= 1000:
        return 3 # Triple points for expert late-game play
    elif score >= 500:
        return 2 # Double points after passing 500 points
    return None # Default 1x multiplier early on

```

How it works:

- **Under 500 points:** Returns None, defaulting to the standard 1x multiplier (100 points per barrel jump).
 - **Between 500 and 999 points:** Returns 2, doubling the bonus to 200 points per jump.
 - **1000+ points:** Returns 3, tripling the bonus to 300 points per jump to reward advanced players who make it to the top platforms.
-